

SPRAWOZDANIE
PROGRAMOWANIE SKRYPTOWE – LABORATORIUM 11

Autor: Piotr Klapetek

Nr albumu: 289863

Data: 3.06.2026

Tytuł: Techniki kryptograficzne

Zad.1 tryhackme

1. część

Zadaniem było odszyfrowanie hasła w pliku zaszyfrowanym

Crack the password with [John The Ripper](#) and rockyou, what's the passphrase for the key?

delicious

✓ Correct Answer



Korzystając z ssh2john najpierw zamieniłem plik który nie jest czystym hashem (posiada nagłówki oraz stopki), wynik przekierowałem na nowy plik.txt

```
(kali@kali)-[~]
$ ssh2john id_rsa_1593558668558.id_rsa > czysty_hash.txt

(kali@kali)-[~]
$ cat czysty_hash.txt
id_rsa_1593558668558.id_rsa:sshng$15168085A04FE869AF092021004350420794941200580ce3420285019a7469e42278a7afab84b4e28c065ce93fef1351bae5499df5fb04ddf510e5e407246e422187db3fbb93931a52762811029b9af30e4c38a56548f3be778
1c7c2b2f594ad0ff7725102ab328bb0c67f0d0e9a3153191dfcd09672256608a5bf044fbf33901849aa2c346ae24bb31d6d0510dd0f01048952a79ce660a97b3123fa539754a0e5ffbfba796c98c17b4ca5eeeee1e1c7a45412e26fef9ba8ed48a15c2b60e23a5a525ee
2451e03c85145d03b7129740b7ec3badad2f012fad21ea8c9ccae7e8ea95e58fe73159db31785f818de90960d3d2a528abddad8337498caa73565042ff8c5dc672d2e58402e3449cf050008e467300220cee35528e718eb25fcd7d6508a2d3dbbe39ed52a445bdf78ad4a9
402b374fc0e741b02f1154b5c29d0602810722c2cfe3b02e0e27f9e41733a35b0c6cf3d83c5f0eafefeb7560d15b5e3a348472794a206d0cfff93649b329ede7e8a2274ce5812b0b0c399cccae532528f8e3442c7e18131780ae5f493b29773d0f728e222470b6
0b2fe0db089ff88793533dc1f95281207405a0a2d284dc096475e765e1a7d43470d490c9d0470b70e09db7943dc1333508e8b0513ed2deccaa51050a496170fd2367b3637e7375891443511168fe1f3292b0644252682865e7da1f1f06a261a52ab155d3a932cc19
76f45c1feaf183ad86c7ce91795fe45395a73268d3c0e228e24d025c997a936fc27bb05992ff4b23e050eda4ae748b14a88c4ff3145f75436100fc840d107eb97e3da3b811a679e373853f8c4431ffc6feecd167f29a75152ad2e099bbcafaae92ae7155684c9175e32fe2
141b07681c37fa4e7910d73b720409e52b0e60f54e0c11b539ad2e0c7de0f52ee20408a192a70581090c18f4237ad0dc20949cac50a40b0207a294f07e1046d14de9051d988ce23ffcf00f4f50eb30191f9081557079c43e992199d137f006e53f89389caa062
cf673ab0b7c099f0c4326a1432ef13074e4cf810e114e0d995faff1baa91efffcce03896724aaba546707d3d3a2074ac3ef078013187fcd034c9d08116d6ce0b3cfba090e0d8997911bae13a370879593af8c3c3d7f5a3809a6870b2a3caca09a479e09780f94
64df6cd743748f3dabbe5e9ac0eb6acd0d8cc4219cbf30bbf8e87fce5b907611bcd1e91a64b1cdab3f26b89f70397e5d0d58e921db7ad69871a6785170b58573eaca996d6cb987210e4d1ea2e098978525b38d80717671d651abea8521768a03c1028570a78514727812
d70f194Acferbaacab0d0d1e5885f7f9acfe7a3770b11a6f42278520ac2fd2105114898fe44b532920841a51e08111be2caa60ab30a3e69c09917fa53271e9400c7994c2486d085a804875c8b470f108e2a2cce0251b04fc27a0f3af083c109745fb17038a80bf520ba
770ca455c3ad1db7b21227cfa0c251fadcf23a70211f0d00e0e0e0a8311d0e0310182521e709ef463246a3a123c1b78e2cae8457aee0fc247380051783b0d095053a232031742bce052db58290d53193392fc55e6d0accedf1f9c2ba98c4da773bcc027f99ab1a448
97280c2d945a0d8a1278206515dd2fa08fc34a4150a0ba35ff0d3d0c2c21cd00e09f774a0741d28534e0c4e43
```

Mając przygotowany plik z czystym hashem, wykorzystałem narzędzie John the Ripper do złamania hasła.

Zastosowałem w tym celu atak słownikowy, używając popularnego, wbudowanego w system Kali Linux słownika

„rockyou.txt”

```
(kali@kali)-[~]
$ cat id_rsa_1593558668558.id_rsa | grep jc40
jc40IChbGadGmmQieKevqwq0DijIZc6T/vE1G65Umd9fvwTd9RDl5AckbkIhh2s/

(kali@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt czysty_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 10 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
delicious (id_rsa_1593558668558.id_rsa)
1g 0:00:00:00 DONE (2026-05-28 14:44) 5.555/s 22222p/s 22222c/s 22222C/s savannah1..summertime
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali@kali)-[~]
$ john --show czysty_hash.txt
id_rsa_1593558668558.id_rsa:delicious

1 password hash cracked, 0 left
```

2. część

Przy pomocy dwóch plików przygotowanych przez moduł (plik message.gpg oraz tryhackme.key)

Plik tryhackme.key posiadał mój klucz prywatny.

You have the private key, and a file encrypted with the public key. Decrypt the file. What's the secret word?

Pineapple

✓ Correct Answer



Klucz prywatny trzeba było zaimportować do systemu przy pomocy narzędzia gpg używając opcje --import

```
(kali㉿kali)-[~]
$ gpg --import gpg/tryhackme.key
gpg: keybox '/home/kali/.gnupg/pubring.kbx' created
gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created
gpg: key FFA4B5252BAEB2E6: public key "TryHackMe (Example Key)" imported
gpg: key FFA4B5252BAEB2E6: secret key imported
gpg: Total number processed: 1
gpg:         imported: 1
gpg:         secret keys read: 1
gpg:         secret keys imported: 1

(kali㉿kali)-[~]
$ cd .gnupg

(kali㉿kali)-[~/gnupg]
$ ls
private-keys-v1.d  pubring.kbx  pubring.kbx~  trustdb.gpg

(kali㉿kali)-[~/gnupg]
$ cat trustdb.gpg
gpgj^
```

Mając importowany klucz prywatny można było odczytać wiadomość

Wykorzystałem również narzędzie gpg z opcją --decrypt oraz wskazując na plik który chciałem odszyfrować

```
(kali㉿kali)-[~]
$ gpg --decrypt gpg/message.gpg
gpg: encrypted with rsa1024 key, ID 2A0A5FDC5081B1C5, created 2020-06-30
"TryHackMe (Example Key)"
You decrypted the file!
The secret word is Pineapple.
```

Zad.2

https://github.com/piotroll1/programowanie_skryptowe/blob/main/szyfr_AES.py

```
szyfr_AES.py x
1  #import biblioteki z modulu crypto.cipher
2  from Crypto.Cipher import AES
3
4  #uzytkownik wpisuje nazwe pliku który chce zaszyfrowac
5  plik=input("wpisz plik, który chcesz zaszyfrowac: ")
6  #wydobreczniona nazwa potrzebna do utworzenia kopii pliku
7  t=plik.split(".")[0]
8
9  def szyfr_AES(plik): 1usage
10     #zapisuje do zmiennej plaintext czysty tekst z pliku podanego przez uzytkownika
11     with open(plik,'r') as f:
12         plaintext=f.read()
13     #zapisanie klucza (AES-128)
14     klucz = b"ToJest16BajtKlucz"
15     szyfr=AES.new(klucz,AES.MODE_EAX)
16     #zapisanie nonce czyli informacji jak zaszyfrowalismy ten plik (pomozę w odszyfrowaniu)
17     klucz_nonce = szyfr.nonce
18     ciphertext=szyfr.encrypt(plaintext.encode("UTF-8"))
19     return ciphertext,klucz_nonce
20     #funkcja zwraca dwie wartosci, zaszyfrowana tresc oraz nonce
21
22
23 #warunek jezeli program jest odpalany z "glownej strony"
24 if __name__ == '__main__':
25     szyfr, nonce = szyfr_AES(plik)
26     #nonce jest w formie zaszyfrowanej, zmieniamy aby bylo w formacie hex
27     nonce_tekst=nonce.hex()
28     wynik=t+"_"+nonce_tekst+"_zaszyfrowany.txt"
29     #zapisanie szyfru do kopii naszego pliku
30     with open(wynik,'wb') as f:
31         f.write(szyfr)
```

Działanie programu. Utworzony został plik

dokument_informacja_jak_zsotal_zaszyfrowany_plik(nonce)_zaszyfrowany.txt.

```
Run szyfr_AES x
C:\Users\piotr\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\piotr\PycharmProjects\PythonProject\szyfr_AES.py
wpisz plik, który chcesz zaszyfrowac: dokument.txt
Process finished with exit code 0

! The file was loaded in a wrong encoding: 'UTF-8' Reload in another encoding
1 wj'4)f0BELACK^00'X!fSTDLER00tDLE00c000[0N
2 ala ma kota
3 kot ma ale
4 alicja kowalska
```

Zad.3

https://github.com/piotroll1/programowanie_skryptowe/blob/main/podpis_cyfrowy.py

```
Project v  podpis_cyfrowy.py  dokument.txt
PythonProject C:\Users\p
  .venv library root
    dokument.txt
    dokument.txt.sig
    podpis_cyfrowy.py
    szyfr_AES.py
  External Libraries
  Scratches and Consoles

1  from Crypto.Signature import pss
2  from Crypto.Hash import SHA256
3  from Crypto.PublicKey import RSA
4
5  #Generujemy klucze
6  klucz=RSA.generate(1024)
7  prywatny=klucz
8  publiczny=klucz.publickey()
9
10 #Zapisywanie klucza publicznego, jest potrzebny aby zweryfikowac pozniej czy nasz dokument podpisany pochodzi od osoby ktora go podpisala
11 with open("klucz_publiczny.pem", "wb") as f:
12     f.write(publiczny.exportKey("PEM"))
13
14 #Odczytywanie oraz hashowanie danych w naszym glownym pliku
15 with open("dokument.txt", "rb") as f:
16     dane=f.read()
17     hash=SHA256.new(dane)
18
19 #tworzymy obiekt ktory bedzie podpisujacy nasz plik
20 obiekt_podpisujacy = pss.new(prywatny)
21 podpis = obiekt_podpisujacy.sign(hash)
22
23 #zapisujemy nasz podpis do pliku .sig
24 with open("dokument.txt.sig", "wb") as f:
25     f.write(podpis)
26
27
```

```
19 #tworzymy obiekt ktory bedzie podpisujacy nasz plik
20 obiekt_podpisujacy = pss.new(prywatny)
21 podpis = obiekt_podpisujacy.sign(hash)
22
23 #zapisujemy nasz podpis do pliku .sig
24 with open("dokument.txt.sig", "wb") as f:
25     f.write(podpis)
26
27 print("Utworzono plik podpisu: dokument.txt.sig. Klucz publiczny zapisano jako klucz_publiczny.pem")
```

Działanie programu

Utworzony został plik podpisany dokument.txt.sig. Klucz publiczny zapisany został w pliku .pem

```
C:\Users\piotr\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\piotr\PycharmProjects\PythonProject\podpis_cyfrowy.py
Utworzono plik podpisu: dokument.txt.sig. Klucz publiczny zapisano jako klucz_publiczny.pem

Process finished with exit code 0
```

```
podpis_cyfrowy.py  dokument.txt  klucz_publiczny.pem
Plugins supporting *.pem files found.

1  -----BEGIN PUBLIC KEY-----
2  MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQD00Gt9AL4LXuLF2K/2ZPsd10h4
3  uTx5Hiho0PgSSVmjbG8h29BXfN1+CfV5MlAuLPNsAG+nr2bWl3XUPPZoAtTkxvKo
4  cT8jBuDmrR/4RrHcmg0AYk1gAvLf2prUi3bgEHNqSfLqCp8h3Nx+VHYa02PkRgI
5  4lFrE0n/+fZUu0emTwIDAQAB
6  -----END PUBLIC KEY-----
```

```
podpis_cyfrowy.py  dokument.txt  klucz_publiczny.pem  dokument.txt.sig
This file was explicitly re-assigned to plain text  Remove association  Edit File

1  r00SUBEM08FS0S
2  S00001300L00DLE00+7-?^0q#0H0ESC0FS000m2SI0CAN00a08EM0<0pVD0BS00_:0DC3=0DC40500000000A500SUB00M
3  W0%0NUL0DebYo0FFn000cF0R0BELF0f00`!]
```